njection attack detection, discrete event logs, semi-Markov processes, feasibility mask, conformal prediction, sequential testing, cyber-physical production systems

Reference-Model Hard Constraints and Semi-Markov Timing Channels for Detecting Injection Attacks on Industrial Discrete-Event Streams

QI YU, School of Cyber Science and Engineering, Southeast University, PR China

WEI LI*, School of Computer Science and Engineering, Southeast University, PR China

In cyber-physical production systems, field devices report their operation as discrete activity events that carry a lifecycle (assignment, start, completion). By injecting or replaying such events on the uplink, an attacker can decouple the log from the physical process. Detectors that threshold only the predicted probability of labels have a structural blind spot here: under a zero-false-alarm constraint they can flag only transitions that the process reference model already forbids, and against forgeries that leave labels intact and merely compress sojourn times, their net detection is zero. We build a detector with three parallel paths. A hard-constraint layer vetoes infeasible events outright using a feasibility mask derived from the reference model; a semi-Markov timing channel produces residuals from grouped log-normal sojourn times; and a structural channel measures whether an activity transition within a process instance is rare. The two statistical channels are calibrated by conformal prediction and accumulate evidence message by message through cumulative-sum (CUSUM) tests, with the false-alarm budget split evenly across the three paths. On the theoretical side, we characterize the blind spot of the label-only detector family in a proposition and derive the critical relative advance $\rho^*(\alpha, \sigma)$, the fraction by which a sojourn must be shortened for a single forgery to be detected with probability one half. We synthesize six attack classes under a reproducible protocol on a real production log from the Fischertechnik Trier smart factory. With a false-alarm budget $\alpha = 0.01$, a delay budget of 10 messages, and the chance-alarm floor subtracted, our net detection is 0.95 for physically infeasible injection and 0.87 for gradual drift, where the label-only ablation scores 0. For early-completion replay at a 20% injection rate it is 0.18, higher than the timed-automaton baseline (significant in a paired test) and not significantly different from the baseline that combines timed automata with a Bayesian network. On state mimicry and message suppression, our method falls below the label-only ablation. The conclusions rest on a single public log and synthetic injections; when thresholds are instead calibrated on the calibration fold, every method exceeds its nominal false-alarm rate.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; • **Computer systems organization** → *Embedded and cyber-physical systems.*

ACM Reference Format:

Qi Yu and Wei Li. 2026. Reference-Model Hard Constraints and Semi-Markov Timing Channels for Detecting Injection Attacks on Industrial Discrete-Event Streams. 1, 1 (October 2026), 24 pages. https://doi.org/XXXXXXX.XXXXXXX

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

1 Introduction

1.1 Background and Motivation

In a cyber-physical production system (CPPS), field devices report their operation as discrete activity events: each event carries a device, an operation (or state label), a timestamp, and a process instance [25, 30]. When the scheduling system assigns a transport device to "move to a given station", the log records the assignment, the start, and the completion, so every activity has a lifecycle. An attacker need not rewrite a control law or compromise firmware: **injecting or replaying these events** on the uplink suffices to decouple the logged trajectory from the physical process. Detection research on industrial cyber-physical systems concentrates on process variables, perimeter protection, and network traffic [12]; lifecycle activity events are rarely treated as an injection target in their own right. We detect whether these events have been injected or replayed; how the scheduler acts on them is outside the scope of the evaluation.

A typical attack replays a "completed" event before the activity has actually finished. The record is entirely legitimate at the label level, yet it rewrites the sojourn time and the context of the events that follow. A detector that only compares predicted and observed labels cannot recognize such an injection: its decision depends only on the (predecessor, label) pair and is identical whether the record is benign or forged (Proposition 5.1).

The same event stream, however, carries three kinds of structure, each matched to a different type of injection. The first is *feasibility*: the reference process model specifies which device may execute which operation and from which predecessor; a log records only the combinations that have occurred and cannot distinguish "never observed" from "not allowed", so these constraints cannot be learned from it. The second is *duration*: activities consume physical time, and an event whose labels are legal but whose timing is wrong can be exposed only by its sojourn time. The third is *workflow structure*: the transition frequencies within a process instance, which change when activities are inserted or deleted.

1.2 Gap in Existing Work

Existing detectors do not exploit these structures because of their methodological premises. Timed automata for production lines [21] model components independently to keep the state space tractable, so cross-component feasibility cannot be expressed; TABOR [18] confines dependencies to a single stage and uses no assignment (command) data; DCTA [4] constrains protocol sessions in smart-grid SCADA rather than the production process; and resilience work at the control layer [9, 24] deals with continuous state estimates rather than discrete activity events. Section 2 gives the detailed comparison.

1.3 Our Approach

We exploit the three kinds of structure with three parallel paths. A hard-constraint layer vetoes infeasible events outright with a feasibility mask derived from the reference model, and additionally checks assignment–completion pairing. A timing channel groups activities by (device, operation) and produces standardized residuals from log-normal sojourn times. A structural channel uses the workflow transition probabilities within a process instance to measure how rare an activity is. The two statistical channels are calibrated by conformal prediction and accumulate evidence message by message through CUSUM tests.

Timing models are not new in themselves: timed automata have been applied to timing anomaly detection on production lines and in SCADA [17, 21], and modelling sojourn times with hidden semi-Markov models is established practice [33]. Our work differs from these in two respects. First, the clock guard of a timed automaton is a binary interval

decision, so a small early report that stays inside the interval is simply accepted; a distributional sojourn model instead turns such same-direction deviations into evidence that can be accumulated. Second, data-driven models flag activity combinations absent from the training data as anomalous and thus mistake new product variants for attacks, whereas reference-model feasibility rejects only combinations that the process does not allow.

1.4 Main Results

We consider six attack classes: naive replay (A1), physically infeasible injection (A2), early-completion replay (A3), state mimicry (A4), gradual drift (A5), and message suppression (A6) (Section 3.4). With a false-alarm budget $\alpha = 0.01$, a delay budget of 10 messages, and the chance-alarm floor (detections produced by random false alarms within the window) subtracted, A2 is covered by the device capability sets of the reference model with a net detection of 0.95, and A5 reaches 0.87 where the label-only ablation scores 0.00. On A3 at a 20% injection rate our method reaches 0.18±0.01, above BUTLA (paired test, Holm $p = 0.031$) but not significantly above the TABOR-style baseline ($p = 0.096$), and the advantage disappears at injection rates of 10% and below. On A4/A6 our method falls below the label-only ablation, and with thresholds calibrated on the calibration fold every method exceeds its nominal false-alarm rate (ours: 0.030). The single-message power curve predicted from the critical relative advance agrees with the measurements (mean absolute deviation about 0.03).

1.5 Contributions

(1) **Problem formalization.** We formalize injection detection on lifecycle activity events (assignment, start, completion), as distinct from the continuous control layer, single process variables, and network packet sessions, and give the system model, the threat model, and reproducible rules for six attack classes (Table 2).
(2) **A hard-constraint layer from the reference-model feasibility mask.** The device-capability check covers every message, is never violated on the cleaned normal data, and rejects none of the 20/508 benign new combinations in the temporal test fold; such a constraint cannot be learned from operational logs.
(3) **Distributional sojourn times with sequential accumulation for label-legal duration forgeries.** Sojourn times are modelled per (device, operation) group, and both statistical channels are calibrated by conformal prediction and accumulated by CUSUM; unlike an interval guard, this accumulates same-direction early reports that stay inside the normal interval (Section 5.2).
(4) **An impossibility result and the critical relative advance.** For label-consistent injections, a detector that thresholds only predicted label probabilities adds nothing beyond the feasibility mask (Proposition 5.1). The critical relative advance $\rho^*(\alpha, \sigma)$, at which single-message detection reaches one half, depends only on the timing variability and the false-alarm budget, and instantiates the limits of stealthy attacks [19] for discrete events without a system matrix.

Sections 2–7 present related work, the problem statement, the detection method, the theoretical analysis, the experiments, and the conclusions, in that order.

2 Related Work

The work closest to ours models industrial processes as timed automata or semi-Markov processes and detects anomalies through timing constraints.

2.1 Timed Automata and Semi-Markov Models

Maier et al. [21] learn timed automata on production lines with the Bottom-Up Timing Learning Algorithm (BUTLA): the component topology is derived from PROFINET adjacency, a separate behaviour model is learned per component, and at detection time an automaton checks whether a symbol is legal, whether its duration lies inside the clock guard, and whether the transition is too rare. Components are split to avoid an explosion of the joint state space, so the method neither checks cross-component consistency nor involves assignment–completion pairing; its clock guard is a binary interval decision, so a small early report inside the guard is accepted; and the tolerance of its probabilistic check is set manually, whereas we let the user specify the false-alarm budget α and calibrate thresholds on benign data.

Lin et al. [17] attach time windows to discrete events in SCADA networks, but their object is message timing rather than process feasibility. Tan and Xi [33] replace the hidden Markov model with a hidden semi-Markov model because the exponential sojourn assumption is unrealistic; sojourn-time modelling as such is therefore not a contribution of this paper.

TABOR [18] combines timed automata with Bayesian networks for one-class classification on the SWaT water-treatment testbed. Its "timing plus dependency" design resembles ours, but each Bayesian-network submodel learns only the dependence of sensors on actuators within the same stage, and the method uses no network or command data, hence no assignment–completion pairing. Although the official SWaT scenarios include cross-stage attacks [10], TABOR can cover them only through the union of independent per-stage detections.

2.2 Packet-Level State Machines and Continuous Control

Goldenberg and Wool [11] characterize Modbus/TCP sessions with deterministic finite automata, and DCTA [4] models IEC-104 communication in smart-grid SCADA as a deterministic counting timed automaton with single-packet, packet-sequence, and timing anomalies. The division resembles our three paths, but DCTA constrains packet attributes and protocol counters of master–terminal sessions and involves neither the production process nor activity semantics. At the control layer, defences for automated guided vehicles such as robust Kalman localization against false data injection [9], and the cyber-physical security line since Mo and Sinopoli [24] on replay, watermarking, and detection delay, deal with continuous state estimates of linear systems rather than discrete activity events.

2.3 Sequential Testing, Conformal Prediction, and Stealthy-Attack Limits

CUSUM and quickest change detection are widely used against false data injection in power grids [16], with delay optimality going back to Lorden and Lai [14, 20]; watermarking and detection-delay bounds [26] and the fundamental limits of stealthy attacks on linear systems [19] complete this toolkit. Conformal prediction provides distribution-free false-alarm control [2, 31] but relies on exchangeability, and coverage degrades under temporal drift [3]. Conformal martingales test exchangeability online [35, 36], and conformal anomaly detection has been tailored to industrial systems through sliding calibration [37] and conditional false-alarm control [13]. We use CUSUM and split conformal calibration without claiming them as contributions (conformal martingales are not used); $\rho^*(\alpha, \sigma)$ is an instance of the stealthy-attack limits in the discrete-event setting, depending only on timing variability rather than on a system matrix.

Table 1. Related work by level. Existing work neither rejects physically infeasible injections through reference-model feasibility nor accumulates duration deviations that stay inside the guard.

Level	Representative	Duration	Feasibility	Assignment pairing
(i) Decomposed production line	Maier/BUTLA [21]	Clock guard	No cross-component check	No
(ii) Single physical process	TABOR [18]	Clock guard	Within-stage dependency only	No
(iii) Packet session	DCTA [4], Modbus DFA [11]	Event timing, counters	No process constraint	No
(iv) Online event injection (ours)	Semi-Markov + reference model	Distributional sojourn	Feasibility mask in hard-constraint layer	Yes

2.4 Conformance Checking and Sequence Anomaly Detection

Process mining checks the conformance of event logs against a reference model [1, 34], and the time perspective can enter the alignment cost [29]. It usually computes fitness offline over complete traces, with no attacker model and no user-specified false-alarm budget. Used as a detector that alarms on any feasibility violation, conformance makes the same decisions as our hard-constraint layer: it covers physically infeasible injection but misses duration attacks that leave labels unchanged (hard-constraint column of Table 4).

Log anomaly detectors train a next-log-key predictor and flag keys outside the top candidates [8, 15, 23]; anomaly detection on process event logs uses autoencoders [27] or next-event prediction [28]. A next-event predictor without time features decides only on (history, label) and therefore belongs to the label-only detector family: replacing the predecessor i in Proposition 5.1 by a finite history leaves the proof unchanged. Learned models with time features are not bound by the proposition and are not included as baselines (Section 7.2).

2.5 Positioning

Table 1 compares related work at four levels. Our work sits at level (iv): online event-injection detection with a reference model. The key difference from level (i) is that its component-decomposition assumption rules out cross-component feasibility; the key difference from level (ii) is that its dependencies are confined to stages. Component decomposition and stage confinement prevent physically infeasible injection (A2) from being rejected by cross-component feasibility, and the binary decision of a clock guard prevents label-legal duration attacks (A3/A5) from being detected by accumulation.

3 Problem Statement and Threat Model

3.1 System Model: The Assignment–Event Loop

A cyber-physical production system consists of a supervisory execution system $\mathcal{C}$ and a set of heterogeneous field devices $\mathcal{D} = \{d_1, \ldots, d_M\}$ connected by an industrial network. Devices receive downlink assignments and report discrete activity events. Our object of study is the integrity of this event stream, not continuous control laws or autonomous coordination among devices.

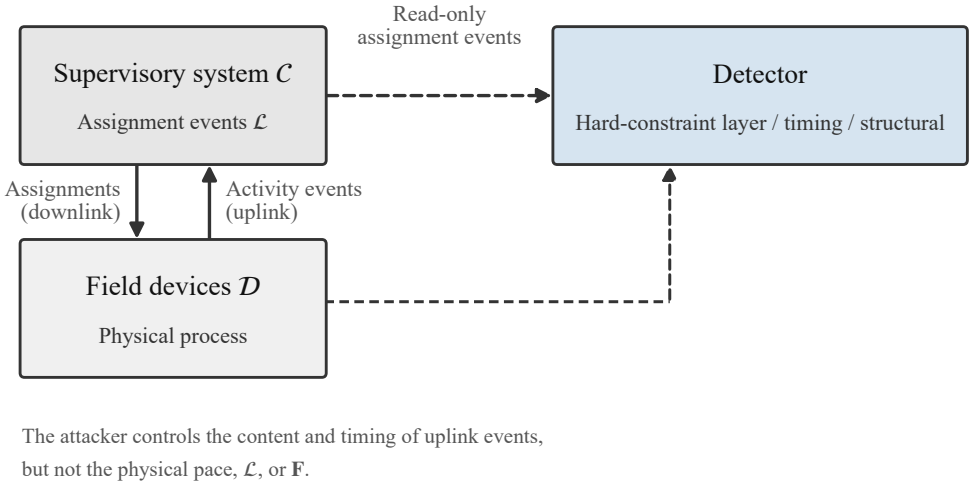


Fig. 1. The assignment/event loop. The detector reads only the assignment events in the same log and the reference-model feasibility mask, and decides online on each uplink event.

Each event is written $m_t = (d_t, o_t, t)$, where $d_t \in \mathcal{D}$ is the device, o_t the operation or state label, and t the timestamp; the event also carries a process-instance identifier (hereafter the *case*), which we omit when no confusion arises. Each event is transmitted over the uplink as one message, and we use "event" and "message" interchangeably. The binary reachability check maintains predecessors per (device, case), and the structural channel maintains predecessors per case; the feasibility mask constrains reachability only within a single workflow instance and never across case boundaries.

Two reporting semantics must be handled separately. Under *event-driven* reporting, an event is sent when the state changes, and the sojourn time is directly observable. Under *periodic polling*, the state is sampled at a fixed period and the same state is reported repeatedly, so only the interval containing the sojourn time is observed (interval censoring). The production log in our main experiments is event-driven; the method keeps a branch for polling, but it is not evaluated. Figure 1 shows the loop structure and where the three detection paths sit.

3.2 Assignment Pairing and the Feasibility Mask

The same log also records the downlink assignments, whose set we denote by $\mathcal{L}$. A completion event without a preceding assignment on the same (device, case) is a causal gap decidable in one step; once process management is integrated with the shop-floor event stream, this pairing can be read directly from the log [30]. The pairing is a causal check, not a duration test (the queueing delay between assignment and start is driven by device contention), and is retained as an implementation detail of the hard-constraint layer (Section 4.2).

Feasibility is derived from the reference process model released with the production line (BPMN in our case) and denoted by the mask F, which contains two kinds of hard constraint. A *unary capability set* specifies whether device d may execute operation o; it does not depend on the predecessor and therefore covers every message. *Binary reachability* specifies whether, on the same case and the same device, operation o may follow the confirmed predecessor operation o'.

The reference model also yields cross-device constraints such as material-token conservation, but without workpiece-level identities these suffer residual benign violations and are not included in the detector (Section 7.2).

3.3 Modelling Assumptions and Threat Model

We make two modelling assumptions. First, the detector reads only uplink events and the assignments in the same log; how the supervisory system dispatches work on their basis is not evaluated. Second, the reference model is consistent with the cleaned normal log, on which the hard-constraint layer has zero violations; the raw log with execution errors does not satisfy this assumption (Table 6).

Attacker capabilities follow the Dolev–Yao model [7]: the attacker acts on the uplink event channel, can eavesdrop on, replay, inject, drop, or delay messages, and may know the state-machine structure and even the transition frequencies. The attacker may know the structure and parameters of the detector (including the σ of each group) but cannot observe its online state (such as the CUSUM statistics and reset times). The attacker cannot compromise the supervisory system or device firmware, cannot change the pace of the physical process, and cannot rewrite assignment events already written to the log or the reference model. The attacker's goal is to decouple the event record from the physical process. Coordinated forgery across multiple devices, which requires aligning several event streams and assignments at once, is outside our scope.

3.4 Attack Classes

Table 2 lists the six injection classes and their reproducible rules. A2 violates the reference model; A3 and A5 leave labels unchanged and only compress sojourn times, sporadically and over consecutive runs respectively. These are the primary targets; A1 serves as the replay reference, and A4/A6 delimit the boundary of the detection capability.

3.5 Detection Problem

Problem 1 (Discrete-event injection detection). *Given a historical log of normal operation only, a reference process model, and a user-specified false-alarm budget α, raise alarms on the online event stream message by message, with the goal that the per-message alarm rate on a purely benign stream does not exceed α. Under exchangeability this goal is guaranteed by conformal calibration; the measured deviation under temporal deployment is reported in Section 6.6. The detector must not use attack labels to tune thresholds. The security goal is* to detect a forgery, or to force its relative advance to stay small*: once the relative advance of a single forgery exceeds $\rho^*(\alpha, \sigma)$, the single-message test detects it with probability greater than one half (Section 5.3).*

We use three evaluation metrics: the net detection rate within the delay budget at a common α (detection rate minus the chance-alarm floor, see Section 6.1); the detection delay in messages (box plots, covering only injections detected within the budget); and the sequential false-alarm rate and alarm intervals on a purely benign stream (ARL_0 is not estimated from them when alarms are too few). The computational constraint is $O(1)$ per message.

4 Detection Method

4.1 Overall Architecture

The production configuration consists of three parallel paths—the hard-constraint layer, the structural channel, and the timing channel—and an alarm is raised when any of them fires. The offline phase uses only normal logs and the reference model: F is derived from the BPMN; transition probabilities and sojourn times are estimated per device and

Table 2. Reproducible rules for the six injection classes. Default injection rate 20%, default $\rho = 0.30$ for A3/A5, three injection seeds; the case is never altered. A4 requires a structural model, without which it degenerates into repeating the current operation and underestimates the attacker. By default the evaluation does not shift the subsequent events of the same instance forward.

ID	Class	Rule
A1	Naive replay	Resend a historical activity of the same device after the current completion time; the historical duration is kept and the label is duplicated.
A2	Physically infeasible injection	Rewrite the operation name in place to a label outside the device's capability set (or to another operation if none exists); subsequent benign events are checked for binary reachability with the forged operation as predecessor (hereafter the cascaded check).
A3	Early-completion replay	Move the completion time earlier in place so that the sojourn time becomes $(1-\rho)\tau$; by default, subsequent events of the same instance are not shifted.
A4	State mimicry	With the model known, insert the most probable next step after the current completion; the device is one that has executed this operation in the log, and the duration is the median of that group.
A5	Gradual drift	Take a run of consecutive activities in time order and compress each sojourn by the same ρ (in contrast to the sporadic sampling of A3).
A6	Message suppression	Delete an entire activity that has not yet started; the label is placed on the next event of the same instance (the observable consequence appears downstream).

operation; the two statistical channels are calibrated by conformal prediction on a held-out calibration fold; and a CUSUM threshold h_i is calibrated for each path on the benign stream so that the path's average run length to false alarm is $\mathrm{ARL}_{0,i} = 1/\alpha_i$, where α_i is the budget assigned to the path (see the next paragraph). The online phase performs a constant number of table lookups and logarithms per message, as shown in Figure 2 and Algorithm 1.

The false-alarm budget is split evenly across the three paths, $\alpha_i = \alpha/3$ ($\mathrm{ARL}_{0,i} = 300$ at $\alpha = 0.01$). Under the logical OR, the overall single-message false-alarm rate is at most α (the Bonferroni bound) and the overall sequential alarm rate at most the sum over the paths; since the hard-constraint layer never fires on normal data, both bounds are conservative. The price of parallel decisions is the split, which every method pays alike: the label-only ablation has a single path and keeps all of α, BUTLA and HSMM have two paths, and the TABOR-style baseline has three, so on an attack where only one path carries signal each of our paths has one third of the budget of the label-only ablation (Section 6.3).

The hard-constraint layer comes first because its decisions produce no false alarms, are interpretable, and keep infeasible samples out of calibration. The structural and timing channels run side by side so that both "wrong label" and "right label at the wrong time" are covered, and both output continuous scores for the final sequential test.

Figure 3 traces two real cases message by message. A2 is vetoed immediately by the unary capability set with a readable alarm reason; the labels of A3 are entirely legal, and its negative residuals, which lie inside the interval guard, cross the threshold after CUSUM accumulation.

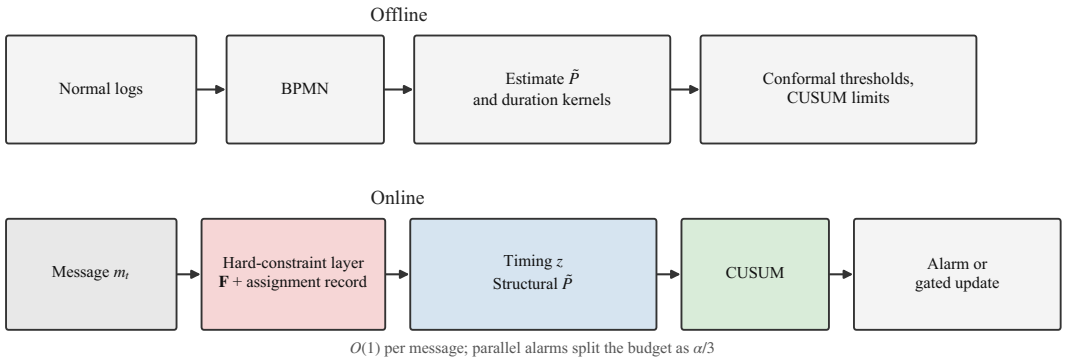


Fig. 2. Offline fitting and online three-path parallel decisions. The hard-constraint layer precedes the statistical channels so that infeasible samples do not contaminate calibration.

Algorithm 1 Online detection of discrete-event injection

Require: event $m = (d, o, t)$; assignments $\mathcal{L}$; mask F; kernels $(\tilde{P}, \mu, \sigma)$; conformal calibrators; thresholds h
Ensure: alarm or accept

if the unary capability, binary reachability, or assignment–completion check fails **then**
    return alarm (hard-constraint reason)
end if
compute the timing residual z and the structural score $\tilde{P}_{o' \to o}$
convert each score to a conformal p-value p with the frozen conformal calibrator
update $S \leftarrow \max(0,\ S - \log p - c)$ for each of the two channels
if $S \geq h$ on either channel **then**
    reset that channel; **return** alarm
end if
if no alarm within the quarantine window **then**
    gated update of (μ, σ) and anchor the state
end if
return accept

4.2 Hard-Constraint Layer

For a message (d, o, t), the hard-constraint layer performs three checks in order; any failure raises an alarm immediately, together with its reason.

(1) Unary: whether (d, o) belongs to the capability set of the reference model.

(2) Binary: if the same (d, case) has a confirmed predecessor o', whether F allows $(o' \to o)$.

(3) Causal: whether the completion event has a preceding assignment event.

The mask is derived automatically from the BPMN released with the dataset: unary capability sets come from the device types (or resource roles) bound to the nodes, and binary reachability is obtained by unfolding the sequence flows and gateways within each workflow, so no transition table has to be written by hand. The unary check covers every message and is never violated on normal data. The value of the reference model is most evident under a temporal split: 20/508 (3.9%) of the benign activities in the test fold are (device, operation) combinations absent from the training fold, far

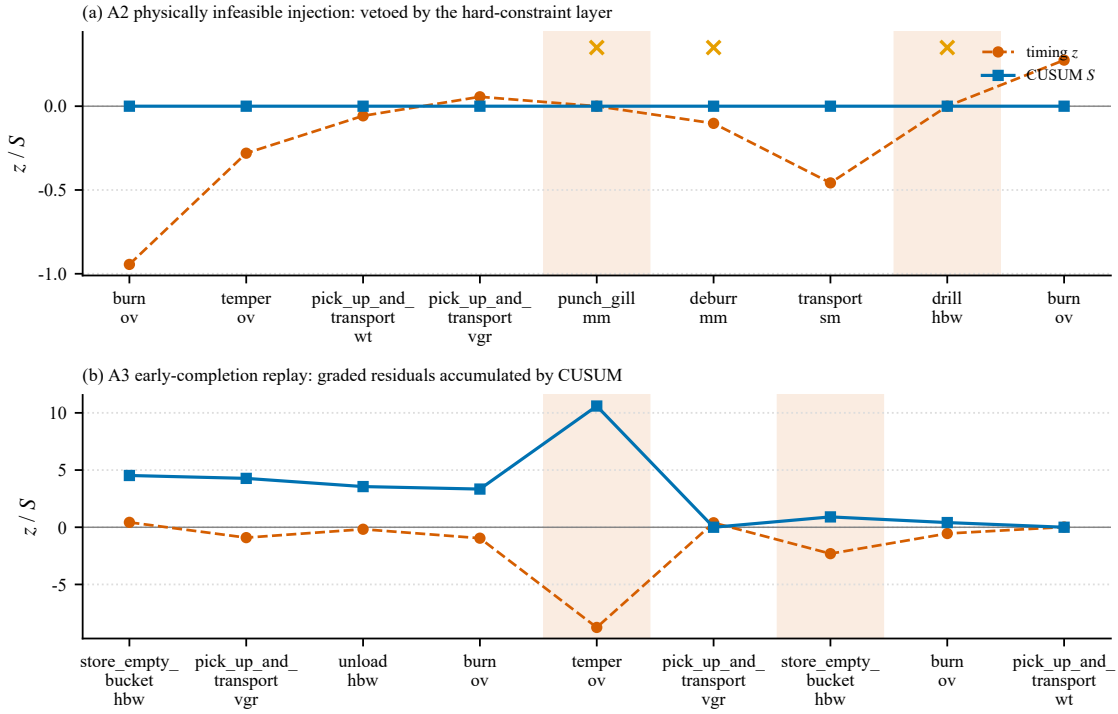


Fig. 3. Message-by-message trace of the hard-constraint veto on A2 and of timing-evidence accumulation on A3. Shaded columns are injected messages; crosses mark hard-constraint alarms.

more than $\alpha = 0.01$, so a rule of "unseen in training means anomalous" would raise many false alarms, whereas the reference model rejects none of them. None of the six injection classes breaks assignment–completion pairing, so the causal check is neither evaluated separately nor counted as a contribution.

4.3 Structural Channel

The structural channel judges whether the current activity is common in the case-level workflow sequence. The chain is kept per case rather than per (device, case): the devices in this log are service endpoints that are invoked, so most per-device event chains have length 1 and their transition probabilities carry little discriminative information. The channel state is the operation name rather than the (device, operation) pair: "the wrong device executing the right operation" is already covered by the unary capability set, and the statistical channel should not additionally assign very small probabilities to unseen combinations, because under a temporal split benign data itself produces new combinations. Transition probabilities are the posterior predictive $\tilde{P}$ of a symmetric Dirichlet prior with concentration 0.5 (Jeffreys-type), which avoids spurious zeros caused by small samples while preserving the true zeros enforced by F.

The score fed to the conformal calibrator is the predicted probability $\tilde{P}_{o' \to o}$ itself, for reasons given in Section 4.5. The first activity of a case has no predecessor, so the structural channel abstains and its score is treated as least anomalous. The channel is relatively sensitive to attacks that change the number of events, such as A4/A6.

4.4 Semi-Markov Timing Channel

The semi-Markov kernel is written $Q_{o'o}(\tau) = P_{o'o}F_{o'o}(\tau)$. Sojourn times are grouped by (device, operation) and modelled as $\log \tau \sim \mathcal{N}(\mu, \sigma^2)$. The difference from a timed automaton is that $F_{o'o}$ is a probability distribution rather than an interval guard (Section 5.2). The location parameter μ is decomposed additively by travel route (source and destination storage positions): routes with at least 8 observations in a group get their own effect, and the rest are pooled into the group's common intercept; unseen routes fall back to the planned duration plus the group's systematic offset. Groups that are absent from the training fold or have fewer than 2 observations, as well as the abstaining groups described below, contribute no timing evidence, and their scores are treated as least anomalous; 56 of the 508 activities in the test fold (11.0%) fall into this category.

Four kinds of observation correspond to four branches: (i) a **state transition** yields the standardized residual $z = (\log \tau - \mu)/\sigma$, tested on the left side only because early reporting is a directional alternative; (ii) a **same-state heartbeat** yields the right-censored survival term $-\log\big(1 - F(\tau)\big)$, which alarms only on overlong sojourns; (iii) a **timeout without message**, raised by a watchdog when the sojourn exceeds the group's 99.9% quantile, uses the same survival term and covers a suppressed completion of a started activity, but not an activity that never started; (iv) **periodic polling** uses the interval-censored term $\log\big[F(\tau + \Delta) - F(\tau)\big]$ and is not evaluated, since the main log is event-driven. The nonconformity score of the timing channel is $-z$ (larger means more anomalous) rather than a parametric p-value, for reasons given in Section 4.5. The σ of manual stations can approach 2; for manual stations and for groups with $\sigma \geq 1$ (where $\rho^* > 90\%$ at $\alpha = 0.01$), the timing channel abstains. Overall detectability is governed by the most variable operation.

4.5 Conformal Calibration and Sequential Accumulation

Both statistical channels apply split conformal prediction [2, 31] on held-out normal samples: for each new message, the rank of its nonconformity score among the calibration scores gives a conformal p-value p_t. When calibration and deployment data are exchangeable, p_t is uniformly distributed on a benign stream, and deciding by $p_t \leq \alpha/3$ guarantees a per-channel false-alarm rate of at most $\alpha/3$. The guarantee requires no Gaussian or Markov assumption, but it does require exchangeability [3], which a time-ordered deployment split does not ensure; Section 6.6 reports the empirical false-alarm rate under such a split. The lowest single-message false-alarm level attainable with a finite calibration set is about $1/(n+1)$, where n is the number of calibration samples; we do not set the single-message α below 10^{-2} and leave lower false-alarm budgets to the ARL_0 of CUSUM.

The nonconformity scores must preserve a monotone order in the tail, so both channels take raw scores as input, not parametric or randomized p-values. The timing channel takes $-z$: once a parametric p-value is truncated at 10^{-12}, benign mismatches and obvious early reports are compressed onto the same value and the tail order is lost. The structural channel takes $\tilde{P}$: a randomized probability integral transform would spread the many ties produced by Dirichlet smoothing over a uniform interval, giving a never-seen transition roughly an even chance of a p-value above α. The conformal p-value has its own randomization term, so ties are broken exactly once, at this step. Section 6.4 reports the measured effect of the choice for the timing channel.

Each channel runs a one-sided CUSUM [16] on its conformal p-values:

$$S_t = \max\big(0,\ S_{t-1} - \log p_t - c\big), \tag{1}$$

where $-\log p_t$ is the evidence contributed by message t; anomalies such as early reporting make p_t small and the evidence large. On a benign stream p_t is uniform and $-\log p_t$ is exponential with mean 1, so the reference value c must exceed 1 for the statistic not to grow without bound on benign data; we use $c = 1.5$. An alarm is raised when S_t crosses the threshold h, which is calibrated per path on the benign stream so that $\mathrm{ARL}_{0,i} = 1/\alpha_i$ ($3/\alpha$ with three paths). The statistic must be reset after an alarm; otherwise it stays above the threshold and every subsequent message raises an alarm. Every baseline that outputs continuous scores uses the same accumulation rule, so that gains are not misattributed to CUSUM itself.

4.6 Gated Update and Complexity

The predecessor state of each chain advances message by message. The duration model slowly updates the (μ, σ) of the corresponding group by an exponential moving average with forgetting factor 0.95; the $\tilde{P}$ of the structural channel is not updated online. Updates must be gated: a message may enter the baseline only if it passes every path and does not fall within the quarantine window (the 20 messages after the most recent alarm); otherwise an attacker could drag the duration baseline towards the forged value through sustained small early reports (Section 6.6). The main-table and ablation experiments use frozen models; the gating and latency experiments follow the same decision protocol with only gated updates enabled. The cost per message is $O(1)$—a constant number of table lookups, logarithms, and hard-constraint checks, independent of the numbers of states and devices—and chains can be processed in parallel (Section 6.6).

5 Theoretical Analysis

5.1 Impossibility of Label-Only Detection for Label-Consistent Injections

Consider a class of detectors that use labels only, hereafter the *label-only detector family*: from the predecessor i it obtains a predictive distribution $\hat{\mathrm{p}}$, encodes the observation as a one-hot vector e_j, and compares $\delta(i, j) = \|\mathrm{e}_j - \hat{\mathrm{p}}\|_2$, or any quantity strictly decreasing in p_j, with an adaptive threshold. Assume that the predictive distribution satisfies $p_j = 0$ if and only if the transition $i \to j$ is forbidden by the feasibility mask F; that is, the predictor uses a smoothed estimate and produces no spurious zeros from insufficient samples (the Dirichlet posterior predictive of Section 4.3 satisfies this condition). Expanding

$$\delta(i, j)^2 = 1 - 2p_j + \|\hat{\mathrm{p}}\|^2 \tag{2}$$

shows that $\|\hat{\mathrm{p}}\|^2$ is constant over all j for a given predecessor i, so δ is merely a monotone transform of p_j and its geometric form adds no information.

Hence δ is a deterministic function of (i, j), regardless of whether the message is benign or injected. Let q_{ij} denote the true frequency with which j follows predecessor i in the benign stream, and FPR_i the false-alarm rate on benign messages that follow predecessor i.

Proposition 5.1 (Impossibility for the label-only detector family). *Suppose that the detector decides on an observation j following predecessor i based only on (i, j), that its decision is a monotone function of p_j, and that $q_{ij} = 0$ if and only if $i \to j$ is forbidden by* F. *Then: (i) under a zero-false-alarm constraint, the transitions that can be flagged are exactly those forbidden by* F*, so for label-consistent injections the family adds nothing beyond the mask; (ii) for injections that leave labels unchanged, the single-message decision on every message equals that on its benign counterpart; if the injection also preserves the message order, the sequential decisions are identical as well, and the net detection is zero regardless of α.*

Proof. (i) If some j with $q_{ij} > 0$ is flagged, every benign occurrence of j raises an alarm, so $\mathrm{FPR}_i \geq q_{ij} > 0$, contradicting the zero-false-alarm constraint; hence only transitions with $q_{ij} = 0$, that is, those forbidden by F, can be flagged. (ii) When labels are unchanged, the pair (i, j) of every message is unchanged and so is its decision; when the order is also unchanged, the score sequence coincides with the benign one message by message, the alarm positions are the same, and any detection comes from chance alarms only. □

The attacker's optimal evasive choice is $\arg\min_j \delta_j = \arg\max_j p_j$, that is, the next step that is most common in the log and thus closest to normal behaviour once forged.

The result does not depend on the particular threshold coefficient: every detector whose decision is a univariate function of the predicted probability belongs to this family. Proposition 5.1(ii) predicts that the first-order Markov ablation has zero net detection on the pure timing attacks A3/A5; Section 6.3 measures −0.01 and 0.00 after subtracting the chance-alarm floor, in agreement with the prediction.

5.2 Distributional Durations versus Clock Guards

A timed automaton assigns a clock guard $[\ell, u]$ to a transition (o', o): a sojourn time $\tau \in [\ell, u]$ is accepted and any other value is rejected [18, 21]. If an attacker shortens a normal sojourn τ to $(1-\rho)\tau$ and $(1-\rho)\tau$ still lies in $[\ell, u]$, the single-step decision is "normal". Because the guard is a binary decision, repeated sojourns that are slightly fast in the same direction are not accumulated.

The semi-Markov channel outputs the continuous residual $z = (\log \tau - \mu)/\sigma$. Early reporting yields $\mathbb{E}[z] = \log(1-\rho)/\sigma < 0$, so the corresponding conformal p-values are systematically small and the mean of the evidence $-\log p_t$ increases. On a benign stream $\mathbb{E}[-\log p_t] = 1 < c$; only when the shift is large enough that $\mathbb{E}[-\log p_t] > c$ does the CUSUM statistic drift upwards and cross h, even if no single observation crosses the single-message threshold on its own. For small shifts the statistic has no positive drift and crosses the threshold only by chance fluctuation, which matches the low detection at small ρ in Figure 8. The difference therefore lies not in whether sojourn times are modelled but in the *form of the evidence*: an interval guard maps early reports inside the interval to zero, whereas a distributional residual maps them to additive increments. A3 is the typical case: its labels are legal and a timed automaton accepts it, yet in the trace of Figure 3(b) its residuals inside the guard are accumulated by CUSUM until the threshold is crossed.

5.3 Critical Relative Advance

Consider a left-sided single-message test that alarms when the residual falls below $-z_{1-\alpha}$. Shortening the sojourn to $(1-\rho)\tau$ shifts the residual distribution to the left; when its mean lies exactly on the threshold, a single forgery is detected with probability $1/2$. An attacker who wants each forgery to evade with probability above one half cannot exceed this critical relative advance ρ^*. It is not a strict upper bound on evasion: forgeries with $\rho < \rho^*$ are still detected with probability below one half, and sequential accumulation raises detection further.

Proposition 5.2 (Critical relative advance). *Let normal sojourns satisfy $\log \tau \sim \mathcal{N}(\mu, \sigma^2)$, and consider a left-sided test of level α with threshold $-z_{1-\alpha}$. The attack shortens the observed duration to $(1-\rho)\tau$. The relative advance at which the post-attack residual mean lies exactly on the threshold is*

$$\rho^*(\alpha, \sigma) = 1 - e^{-\sigma z_{1-\alpha}}. \tag{3}$$

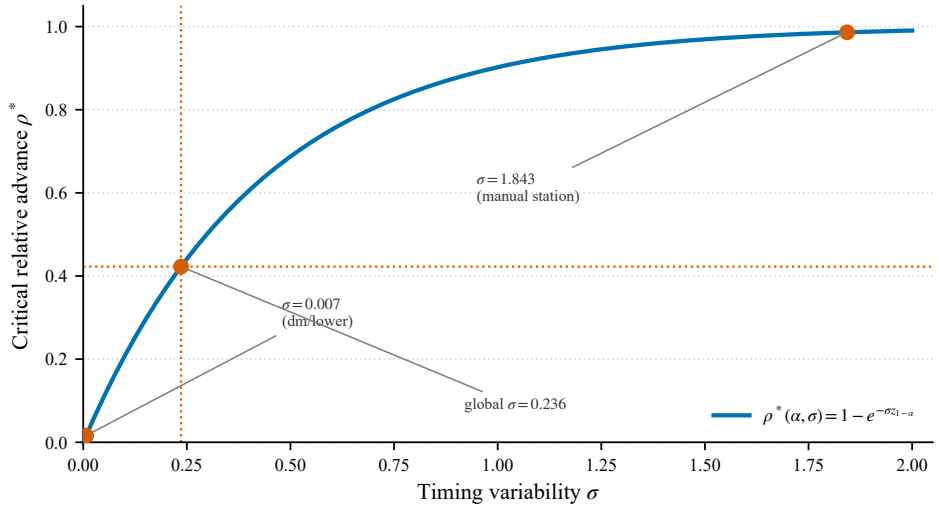


Fig. 4. ρ^* as a function of σ for a one-sided test at $\alpha = 0.01$. Marked points are a low-variability operation, the globally weighted σ, and a manual station.

If the test acts on the mean of k independent forged residuals, whose standard deviation shrinks as $1/\sqrt{k}$, then

$$\rho_k^*(\alpha, \sigma) \approx 1 - e^{-\sigma z_{1-\alpha}/\sqrt{k}}. \tag{4}$$

Proof. The log-domain shift is $\Delta = \log(1 - \rho)$, which moves the standardized mean by Δ/σ. Setting $\Delta/\sigma = -z_{1-\alpha}$ gives $\rho = 1 - e^{-\sigma z_{1-\alpha}}$. Averaging k independent residuals replaces σ by $\sigma/\sqrt{k}$, which yields Eq. (4). □

Eq. (3) involves only the timing variability and the false-alarm budget, not a system matrix. It instantiates the fundamental limits of stealthy attacks [19] in the discrete-event setting; the form is not new, but the instantiated quantity is the relative advance rather than the energy of a continuous state deviation. Our experiments validate single injections only, so the $\sqrt{k}$ tightening of Eq. (4) is not presented as a validated result.

The one-half point refers to a *single* Gaussian. The σ of the operations on a line are highly heterogeneous, and low-variability groups are detected at relative advances far below the global ρ^*, so the detection curve mixed over operations (the mixture power curve) can lie clearly above 1/2 already for $\rho < \rho^*$ (validated in Section 6.5). Figure 4 shows ρ^* as a function of σ: the global $\sigma = 0.236$ corresponds to 42.2% at $\alpha = 0.01$ (47.3% at the $\alpha/3$ of the three-path configuration); as the σ of manual stations approaches 2, ρ^* tends to 1, so the detectability of the line is governed by its most variable operation.

6 Experiments and Results

6.1 Experimental Setup

The main data are the Fischertechnik smart-factory IoT event log of the University of Trier [22], chosen because it is one of the few public activity logs that provide both a BPMN reference model and the assignment–execution–completion lifecycle, not because its scale is representative. Process-variable datasets such as HAI cannot instantiate assignments and activities [32], and public industrial-control datasets are likewise dominated by process variables and network traffic [5, 6]; we therefore report no cross-dataset results. The cleaned log contains 3,062 activities on about a dozen

Table 3. Reimplementation protocol for the main baselines. The comparison is against the structural constraints of the original papers, not against official binaries.

Method	Retained	Approximation on this log
B1 BUTLA-style [21]	One automaton learned per component; no cross-component check; duration as a deviation tolerance	Component taken as device; tolerance replaced by the empirical quantile at the same α, so that differences in false alarms are not counted as differences between methods
B2 TABOR-style [18]	Within-stage dependencies; no assignment (command) data	Stage taken as device; within-stage joint as $P(o)\prod P(v \mid o)$ (compared with the saturated joint below the table)
B3 HSMM [33]	Semi-Markov model with explicit durations	Observations are the operation and $\log \tau$; $K \in \{2, 3, 4, 6\}$, with $K = 6$ selected by held-out likelihood

invoked service endpoints. Diversity within the log serves only for stress tests, not as a second external dataset: 16 released BPMN models, the official error-containing version (NTP clock drift, duplicated and missing records), and five workflows in the temporal test fold that are unseen in training.

Public activity logs contain no injection labels, so we map replay, injection, and dropping in the Dolev–Yao model to the log fields (Table 2); the injector is released with the code. The data are split in time order into fit (1,951 activities), calibration (603), and test (508 benign events) folds, and injections are applied only to the test fold, at a default rate of 20% (swept over 5%/10%/20%/40%) and $\rho = 0.30$. The sample structure is 1 log × 1 temporal split × 6 attack classes × 3 injection seeds: the seeds $(0, 1, 2)$ determine only the injection positions, the fitting seed is fixed at 0, each class has 297 injections over the three seeds (290 for A1, 303 for A2, 201 for A6), and under a given seed all methods face the same attacked stream. The relative-advance sweep (Section 6.5) uses a calibration set stratified by (device, operation) group ($n_{\text{cal}} = 701$). All experiments run on a single machine (Intel Xeon W-1270, 64 GB of memory) with Python 3.7.6, NumPy 1.18.1, and SciPy 1.4.1.

The three main baselines are reimplemented from the structural constraints of the original papers (Table 3), since neither BUTLA nor TABOR has a released implementation that accepts this log format and HSMM is a model class rather than an industrial detector. Every method that outputs continuous scores uses the same CUSUM ($c = 1.5$, delay budget of 10 messages, reset after alarm) and splits its budget evenly across its sub-detectors, each calibrated for $\text{ARL}_0 = 1/\alpha_i$. To compare methods at the same false-alarm level, the main-table thresholds are calibrated on the uninjected test fold; this uses no attack labels but borrows the benign test distribution and is thus idealized, so we also rerun every method with thresholds from the calibration fold (Section 6.3). All methods are fitted on the fit fold only. In the main table our three raw scores and the baseline scores pass through the same decision procedure (empirical p-values and the CUSUM above), bypassing the conformal calibrator so that two threshold mechanisms are not mixed; per-channel conformal false-alarm rates are given in Section 6.6. The label-only first-order Markov likelihood is an ablation that removes the time dimension, not a prior method, and the reference-model conformance proxy coincides with the hard-constraint layer (Table 4).

The within-stage structure learned in the original paper lies between the naive factorization of Table 3 and the saturated joint $P(o, v_1, \ldots, v_m)$; with the saturated joint, B2's six-class mean changes from 0.21 to 0.22 and its A3 result from 0.10 to 0.08, and our method remains above this variant on all six classes.

Table 4. Single-message detection rates of the three production paths on the six injection classes, with the false-alarm rate on the same benign events in parentheses. Each path decides on its own at $\alpha = 0.01$ with frozen models; test fold of 508 events, injection rate 20%, mean over three injection seeds. The hard-constraint value of 0.06 on A2 comes from the cascaded check: the benign event after an injection has the forged operation as predecessor and fails binary reachability, i.e. the attack is detected but attributed to the following event; it is therefore not a purely benign false alarm.

ID	Class	Hard constraint F	Structural	Timing
A1	Naive replay	0.22 (0.00)	0.21 (0.02)	0.03 (0.02)
A2	Physically infeasible injection	0.98 (0.06)	0.29 (0.02)	0.00 (0.02)
A3	Early-completion replay	0.00 (0.00)	0.02 (0.01)	0.43 (0.02)
A4	State mimicry (known model)	0.03 (0.00)	0.01 (0.01)	0.00 (0.02)
A5	Gradual drift	0.00 (0.00)	0.01 (0.01)	0.48 (0.03)
A6	Message suppression	0.00 (0.00)	0.16 (0.01)	0.04 (0.02)

Under the rule "any alarm in the window counts as a detection", random false alarms yield some detection even for a detector without real capability. This chance-alarm floor, $1 - (1 - \mathrm{FPR})^{B+1}$ with delay budget $B = 10$, is subtracted from all main results.

Methods are paired per injection instance. For each attack class we apply a two-sided exact McNemar test to our method against each competitor, with 95% bootstrap intervals (2,000 resamples over injections) for the difference in detection rate and Holm correction over the $6 \times 4 = 24$ comparisons. The test uses raw detections without floor subtraction, and treating injections in the same stream as independent is an approximation.

6.2 Single-Channel Coverage Diagnosis

Before the end-to-end comparison, Table 4 gives the single-message detection rate of each of the three paths, to show which path is responsible for which attack. A2 is rejected by the unary capability set, and the timing channel is insensitive to it. A3/A5 leave labels unchanged and only compress sojourn times; the hard-constraint layer and the structural channel detect almost nothing, and the timing channel is the only one with substantial detection. A1 keeps historical durations, so the timing channel gains nothing and detection is carried by the hard-constraint layer and the structural channel. A4 inserts the most probable next step according to the transition model, and the single-message detection of all three paths is close to the false-alarm level. A6 deletes an entire activity that has not yet started, so the timeout watchdog is never triggered, and the structural channel sees only a weak signal downstream. Being a single-channel, single-message diagnosis without budget splitting or floor subtraction, Table 4 is not comparable with the end-to-end net detection of Table 5; the single-message timing detection of 0.43 on A3 and the 0.18 there do not conflict.

6.3 Comparison with Timed Automata and Semi-Markov Models

Table 5 gives the net detection rates at $\alpha = 0.01$. Our method averages 0.48 over the six attack classes, against 0.17/0.21/0.11 for B1/B2/B3. We discuss three groups of attacks in turn.

First, pure timing attacks. On A3/A5 the label-only ablation scores −0.01 and 0.00; its score stream coincides with the benign one message by message, as Proposition 5.1(ii) predicts. On A5 our method reaches 0.87; timed-automaton baselines also capture cumulative drift (0.59/0.48) but stay below the per-group left-tail test. On A3 our method reaches 0.18±0.01 against 0.10 for both B1 and B2; the paired difference is 0.067 against B1 (95% interval $[0.020, 0.114]$, Holm-corrected $p = 0.031$) and 0.064 against B2 ($[0.010, 0.118]$, $p = 0.096$, not significant). The modest gain comes from

Table 5. Net detection rates at $\alpha = 0.01$ with a delay budget of 10 messages and the chance-alarm floor subtracted. Mean ± population standard deviation over three injection seeds, 201–303 injections per attack class in total. Label-only is the Markov ablation. Sequential false-alarm rates in the attacked streams (nominal 0.01): B1 0.006, B2 0.006, B3 0.008, label-only 0.008, ours 0.004. Paired tests and results with thresholds calibrated on the calibration fold are reported in Section 6.3.

Attack	B1 BUTLA	B2 TABOR-style	B3 HSMM	Label-only	Ours
A1 Naive replay	0.04±0.01	0.04±0.01	−0.01±0.03	0.48±0.04	0.55±0.04
A2 Infeasible injection	0.37±0.05	0.67±0.03	0.31±0.04	0.76±0.04	0.95±0.00
A3 Early-completion replay	0.10±0.03	0.10±0.04	0.06±0.04	−0.01±0.02	0.18±0.01
A4 State mimicry	−0.04±0.01	−0.01±0.01	0.02±0.01	0.23±0.04	0.17±0.03
A5 Gradual drift	0.59±0.09	0.48±0.07	0.25±0.08	0.00±0.12	0.87±0.06
A6 Message suppression	−0.05±0.01	−0.01±0.03	0.05±0.02	0.28±0.08	0.14±0.06
Mean	0.17	0.21	0.11	0.29	0.48

accumulating sporadic early reports under distributional residuals. B3 reaches only 0.06, so the gain is not merely "semi-Markov versus interval guard": the HSMM duration kernel is tied to hidden states, which absorb an early report with an unchanged label as a slightly shorter stay.

Second, replay and infeasible injection. On A1/A2 our method reaches 0.55/0.95, above all three baselines (Holm-corrected $p < 10^{-20}$ in every case). A2 is covered by the unary capability set, which the within-stage dependencies of B2 cannot replace: B2 assigns extreme scores to never-executed (device, operation) combinations, but such combinations already occur in the benign test fold (Section 4.2). B3, whose EM likelihood converges monotonically with a non-degenerate duration kernel, reaches only 0.31 on A2, consistent with the absence of feasibility constraints in a hidden-state model.

Third, attacks that change the number of events. On A4/A6 our method reaches 0.17/0.14, below the label-only ablation's 0.23/0.28 (Holm-corrected $p = 0.001$ and $p < 10^{-6}$). These attacks are handled mainly by the structural channel, which receives $\alpha/3$ while the label-only ablation keeps all of α; the budget split is consistent with the gap, which we report as a cost of the multi-channel design rather than as a main conclusion. At $\alpha = 0.05$ the gap persists (0.18/0.19 versus 0.41/0.45), so it is not a calibration-resolution artefact; at that level our method also falls below the label-only ablation on A1 (0.47 versus 0.61).

With thresholds calibrated on the calibration fold instead (Section 6.1), every method exceeds the nominal sequential false-alarm rate of 0.01 on the test fold: B1 0.043, B2 0.138, B3 0.028, label-only 0.018, ours 0.030, consistent with the per-channel results of Section 6.6. With the false-alarm rates no longer aligned, the net detections (each with the floor of its own measured false-alarm rate subtracted) are only qualitative: our method is still the highest on A3/A5 (0.45/0.71) and in the six-class mean (0.44, against 0.13, −0.11, 0.18, and 0.32 for B1, B2, B3, and label-only), but falls below the label-only ablation on A2 (0.72 versus 0.76). Taken together, the advantage on duration forgery holds against all three baselines on A5, and on A3 it is significant only against B1/B3.

Figure 5 shows box plots of the detection delay, counting only injections detected within the delay budget. On the purely benign stream of the test fold (508 events) each method raises only 2–4 alarms (ours: 2, a sequential false-alarm rate of 0.004), too few to estimate ARL_0. Figure 6 sweeps the injection rate from 5% to 40%: on A2 our method is the highest at every rate, with a lead that narrows as the rate grows; on A5 it is slightly below B1 only at 10% (0.67 versus 0.69); on A3 it exceeds the timed-automaton baselines only at 20% and 40%; on A4 it is below the label-only ablation at every rate, and on A6 it exceeds the latter only at 5%.

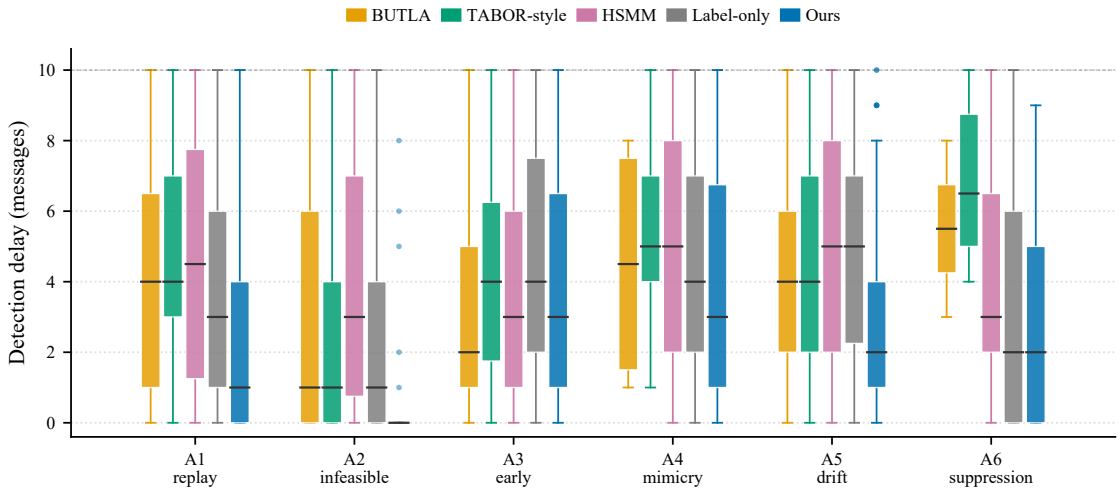


Fig. 5. Detection delays (in messages) of each method on the six attack classes at $\alpha = 0.01$ with a delay budget of 10 messages. Boxes include only detections within the budget; for undetected injections see the net detection rates.

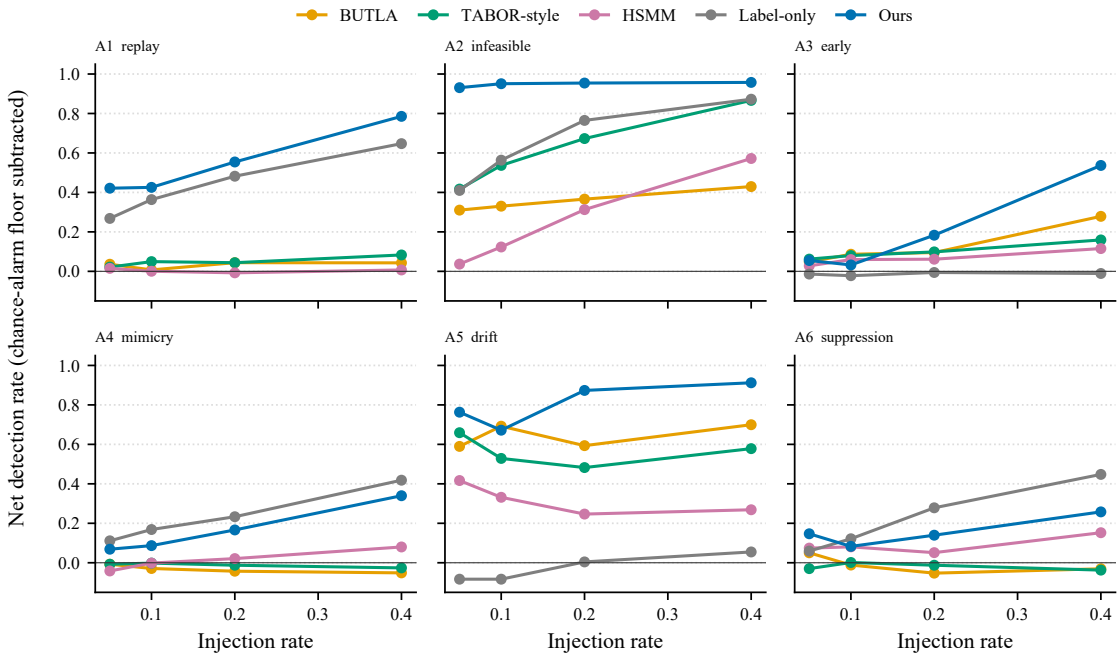


Fig. 6. Net detection rate versus injection rate (5%–40%). One panel per attack class, showing whether the conclusions break down as attack density increases.

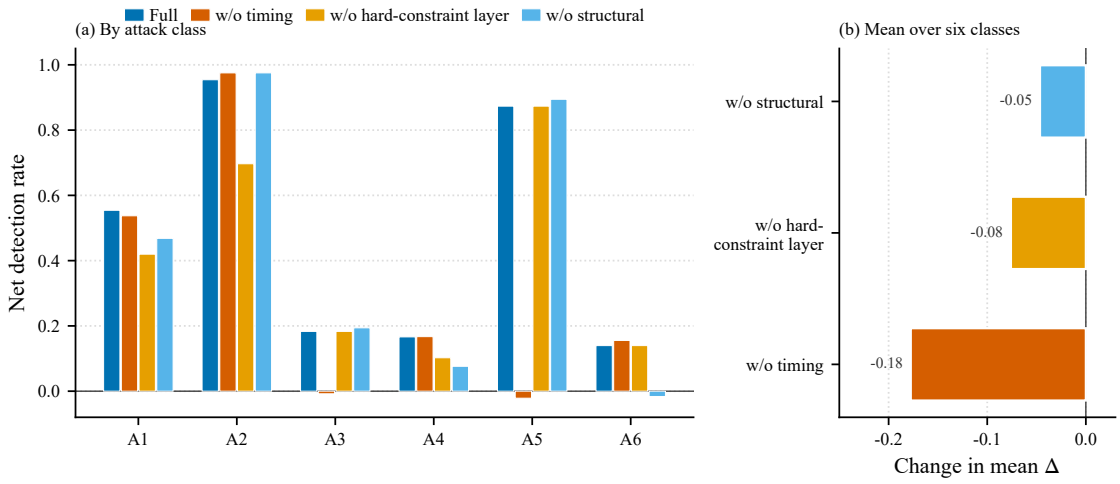


Fig. 7. Ablation of the three production paths ($\alpha = 0.01$, fixed per-path false-alarm budget, mean over three injection seeds, chance-alarm floor subtracted).

6.4 Ablation

The ablation uses the main-table procedure at $\alpha = 0.01$ and *fixes each path's budget*, so that removing a path gives no extra budget to the others (Figure 7). From the full mean of 0.48, removing the timing channel costs 0.18 and drives A3/A5 to zero ($0.18 \to -0.01$, $0.87 \to -0.02$): the remaining two paths read labels only, which corroborates the label-only result on the same classes. Removing the hard-constraint layer costs 0.08, mainly on A2 ($0.95 \to 0.70$) and A1 ($0.55 \to 0.42$); removing the structural channel costs 0.05, mainly on A6 ($0.14 \to -0.02$) and A4 ($0.17 \to 0.08$). At $\alpha = 0.05$ the three drops are 0.16, 0.04, and 0.03, in the same order. Only the timing channel is irreplaceable: it is the sole source of detection on A3/A5.

Three design choices were also settled by measurement. Replacing the conformal one-sided threshold by a Gaussian quantile raises the empirical test false-alarm rate from 0.0126 to 0.0168 (nominal 0.01, stratified calibration set of Section 6.5). Feeding the timing channel a truncated parametric p-value instead of $-z$ lowers the single-message detection on A3 from 0.43 to 0.06 (basis of Table 4). Material-token conservation, with too high a benign violation rate and zero net end-to-end contribution, is excluded and consumes no budget (Section 7.2).

6.5 Relative-Advance Sweep

We apply the injection $\tau \mapsto (1-\rho)\tau$ to real sojourn times. The ρ^* of Proposition 5.2 uses the global $\sigma = 0.236$ of the fit fold (not conditioned on covariates), giving $\rho^* = 42.2\%$ for a one-sided test at $\alpha = 0.01$. The theory curve in Figure 8 is the mixture prediction over operation groups (Section 5.3); their mean absolute deviation from the measurements over the whole range is 0.027 (95% interval $[0.018, 0.035]$ from 2000 bootstrap resamples drawn independently from the calibration and test folds; 0.012–0.031 over 20 fold-split seeds). At $\rho = 0.30 < \rho^*$ the measured single-message detection already reaches 0.52, and at $\rho = 0.40 \approx \rho^*$ it is 0.74, whereas a single Gaussian would give only $1/2$ at ρ^*; this is exactly what the mixture power curve predicts.

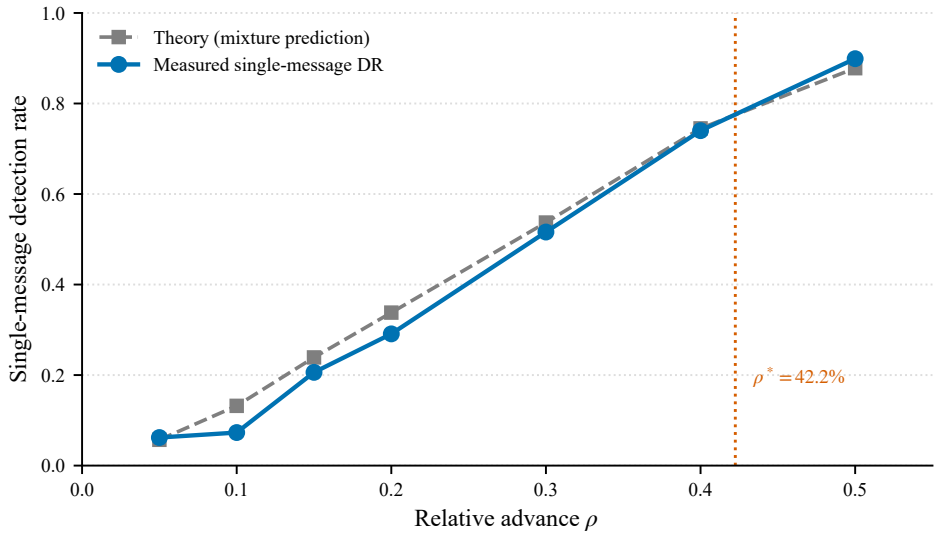


Fig. 8. Single-message detection rate versus relative advance for a one-sided test at $\alpha = 0.01$ (stratified calibration set $n_{\text{cal}} = 701$, 715 test events). The theory curve is the group-mixture prediction; the vertical line marks $\rho^* = 42.2\%$ at the global $\sigma = 0.236$.

CUSUM turns weak signals into usable detection capability. In this experiment it is driven by parametric left-tail p-values on the stratified calibration set, with a threshold corresponding to $\text{ARL}_0 = 500$ and a delay budget of 40 messages. At $\rho = 0.15$ the single-message detection is 0.21 and rises to 0.55 after accumulation, with a median delay of 15 messages; at $\rho = 0.20$ it rises to 0.85; at $\rho = 0.30$ it is 0.93, with a median delay of 7 messages (Figure 9). Detection saturates at about 0.96; the undetected remainder comes from groups with very large σ, consistent with overall detectability being governed by the most variable operation.

6.6 Deployment Conditions and Scope

This subsection reports four sets of data that bear on deployment.

First, false alarms under a temporal split (Table 6, upper part). With fitting and testing split 75%/25% by case (about 636 test messages), the per-channel conformal false-alarm rates stay close to nominal under 5 random splits, but under the temporal split at $\alpha = 0.01$ the timing channel rises to 0.024 and the structural channel to 0.012 (0.098 and 0.064 at $\alpha = 0.05$); the temporal split admits only one partition, so its 5 seeds vary only the fitting and calibration randomization. The median Jaccard similarity of the transition sets of the 16 released workflows is only 0.324, so new product types must be recalibrated or calibrated in strata by type. These per-channel single-message rates differ in definition from the sequential false-alarm rate of 0.004 in Section 6.3; with sequential thresholds from the calibration fold, ours rises to 0.030, in the same direction.

Second, stress tests within the log (Table 6, lower part). The timing false-alarm rate is 0.112 on the official error-containing version, caused by data-acquisition defects rather than attacks, and 0.067 on unseen workflows, while the hard-constraint layer stays at 0.003 and 0. Product changeover is a distribution shift: the timing channel must be recalibrated per product type, whereas the hard-constraint layer need not be.

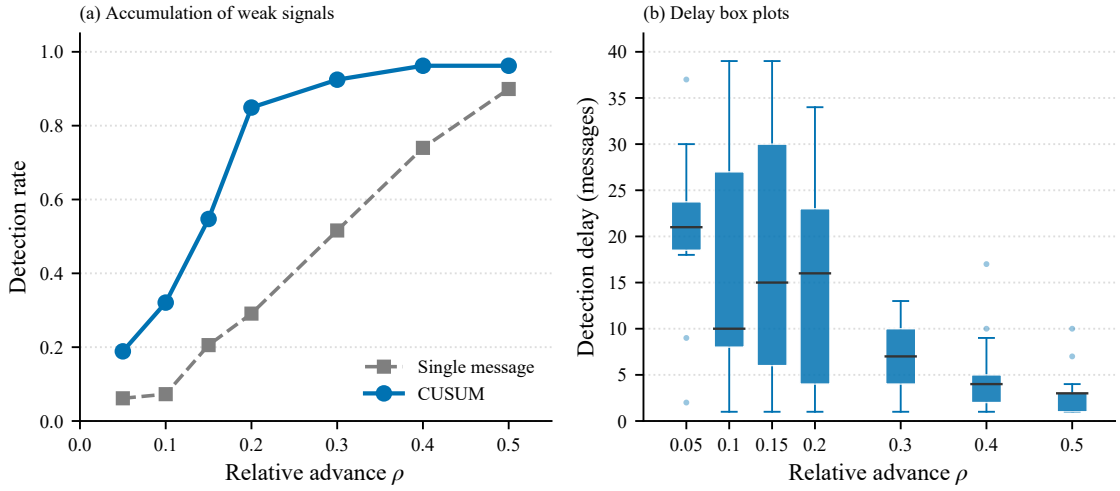


Fig. 9. Power of CUSUM relative to the single-message threshold, and box plots of the detection delay. Driven by parametric left-tail p-values on the stratified calibration set, threshold for $\mathrm{ARL}_0 = 500$, delay budget of 40 messages, 53 test substreams. Boxes include only injections detected within the budget; undetected injections are censored (see the net detection rates in the main table).

Third, recalibration of the duration kernel. Doubling the σ of the fit fold lowers the single-message and CUSUM detection at $\rho = 0.30$ from 0.52 and 0.94 to 0.22 and 0.45; at $\sigma \approx 0.96$ only 0.05 single-message detection remains. A self-built simulated line shows that the kernel must be recalibrated per scenario. Its three scenarios share the operation vocabulary and F: I has 2 automated guided vehicles and 1 robot arm with $\sigma = 0.12$ and short travel distances; II has 4 vehicles and 2 arms with $\sigma = 0.12$ and long distances; III has the layout of I with $\sigma = 0.35$; each has 69–90 benign events and 13–18 A3 injections ($\alpha = 0.01$, $\rho = 0.30$). With each scenario's own kernel, the single-message A3 detection of the timing channel is 0.462, 0.556, and 0.308 on I, II, and III. With the kernel of I it drops to 0.056 on II (1/18 versus 10/18) and to 0.385 on III, where the timing false-alarm rate rises from 0 to 0.101; the reverse transfer II→I raises it to 0.493. The hard-constraint layer has zero false alarms in every setting, since all scenarios share F.

Fourth, gating and computational cost. Under 200 sustained early reports with $\rho = 0.30$, disabling the gate lets 26.4% of the forged advance enter the baseline (all 200/200 updates take effect), whereas enabling it reduces this to 1.4% (3/200). The median per-event latency on a single core is 123 μs ($p_{95} = 191$ μs, $n = 1{,}524$) and does not grow when the scale is increased by replicating device names; we did not measure on a dedicated controller.

7 Conclusion

7.1 Conclusions and Contributions

We have formalized injection detection on lifecycle activity events. Theoretically, Proposition 5.1 shows that detectors which threshold only predicted label probabilities add nothing beyond the feasibility mask for label-consistent injections under a zero-false-alarm constraint, and Proposition 5.2 gives the critical relative advance $\rho^*(\alpha, \sigma)$. Methodologically, we combine a hard-constraint layer from the reference-model feasibility mask with a timing channel of distributional sojourn times per (device, operation), both statistical channels being calibrated conformally and accumulated by CUSUM; unlike decomposed timed automata and within-stage dependency models, early reports inside the guard can be accumulated and feasibility comes from the reference model rather than the log. Empirically, physically

Table 6. Per-path empirical false-alarm rates without injection at $\alpha = 0.01$. Upper part: conformal rates under case-level 75%/25% splits (random: 5 splits; temporal: one split, 5 seeds), mean ± standard deviation; the hard-constraint layer was not measured in this experiment. Lower part: stress tests within the same Trier log; the error-containing version is released by the original authors, and seen and unseen workflows are obtained by splitting the temporal test fold.

Setting	n	Hard constraint	Timing	Structural
Random case split	≈636	–	0.006±0.005	0.008±0.006
Temporal case split	≈636	–	0.024±0.001	0.012±0.001
Cleaned temporal test fold	508	0.000	0.024	0.012
Official error-containing full log	2,778	0.003	0.112	0.011
Seen workflows	329	0.000	0.000	0.012
Unseen workflows	179	0.000	0.067	0.006

infeasible injection is covered by the mask (net detection 0.95), gradual drift reaches 0.87 where the label-only ablation scores zero, early-completion replay at a 20% injection rate reaches 0.18 (above BUTLA, not significantly different from the TABOR-style baseline), and the mixture power curve agrees with the measurements. Reference-model feasibility and distributional sojourn times are thus two kinds of evidence missing from label-only detection; the disadvantage on A4/A6 shows that they do not replace workflow structure.

7.2 Limitations

(1) State mimicry and message suppression lack a dedicated detection path. On A4/A6 our detection is below that of the label-only ablation. There are two possible causes whose effects have not been separated: after the three-way split the structural channel receives only $\alpha/3$, and its state is the operation name, which is insensitive to the device. This limitation qualifies the comparison of "six-class means" but does not affect the claims about A2 and A3/A5.

(2) Without workpiece identities, material-flow invariants cannot serve as reliable constraints. The benign violation rate q rises from 0.0054 on the training fold to 0.047 on the deployment stream, which caps single-message detection at only 0.21 for $\alpha = 0.01$. Approximate cross-case workpiece identities lower q to 2.3%, still above α, and end-to-end performance is the same as without the constraint, so the token constraint is not included in the detector. This limitation does not affect the three claims of this paper; it only means that the hard-constraint layer cannot yet cover cross-device material-flow forgery.

(3) The main evaluation uses a single public log, synthetic injections, and reimplemented baselines. Attacks are synthesized by a reproducible injector, the baselines are reimplemented from the structural constraints of the original papers rather than taken from official code, and no learned sequence model with time features is compared. Within-log stress tests cannot substitute for a second external dataset; under a temporal split the conformal guarantee no longer holds automatically, and duration kernels cannot be transferred across scenarios directly (Section 6.6). This limitation threatens the external validity of the results on A2 and A3/A5, but does not affect Proposition 5.1.

7.3 Future Work

(1) Material-token constraints on lines with workpiece identifiers (Limitation (2)). If the benign violation rate in deployment can be brought down to the order of α, token conservation could extend the hard-constraint layer to cross-device material-flow forgery.

(2) Label-free allocation of the false-alarm budget across channels (Limitation (1)). The even $\alpha/3$ split may underlie the disadvantage on A4/A6; any allocation rule must not tune thresholds with attack labels.

(3) External validation and online recalibration (Limitation (3)). This requires a second public log with both BPMN and the activity lifecycle, official baseline implementations, learned sequence baselines with time features, and gate-protected adaptive updating of the duration kernel per product type, since the self-built scenarios show that kernels do not transfer while the feasibility mask does.

8 Data Availability

The Fischertechnik Trier smart-factory event log is released by its original authors [22]; a copy of the data is available on Figshare at `10.6084/m9.figshare.20130794`. The injection protocol, baseline reimplementations, diagnostic scripts, and evaluation archives are in the accompanying code repository, available from the corresponding author during review and to be made public upon acceptance. The main tables and box plots are read from JSON archives, and no quantile is filled in by hand. Paper-level CSV files are aggregated from the archives by `py -m tools.export_experiments`. The self-built scenarios I/II/III (configurations A/B/C in the code) are regenerated from seeds by `algorithm/plant.py`. HAI is used only to check the data dictionary and is not imported into the CSV files.

Acknowledgments

We thank the University of Trier for releasing an IoT event log with a BPMN reference model, which made it possible for the feasibility mask to come from an external reference model rather than being learned from the log.

References

[1] Arya Adriansyah, Boudewijn F. van Dongen, and Wil M. P. van der Aalst. 2011. Conformance Checking Using Cost-Based Fitness Analysis. In *Proceedings of the 15th IEEE International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE Computer Society, Helsinki, Finland, 55–64. doi:10.1109/EDOC.2011.12

[2] Anastasios N. Angelopoulos and Stephen Bates. 2023. Conformal Prediction: A Gentle Introduction. *Foundations and Trends in Machine Learning* 16, 4 (2023), 494–591. doi:10.1561/2200000101

[3] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal Prediction beyond Exchangeability. *The Annals of Statistics* 51, 2 (2023), 816–845. doi:10.1214/23-AOS2276

[4] Nisha Kumari Barsha and Neminath Hubballi. 2024. Anomaly Detection in SCADA Systems: A State Transition Modeling. *IEEE Transactions on Network and Service Management* 21, 3 (2024), 3511–3521. doi:10.1109/TNSM.2024.3373881

[5] Mauro Conti, Denis Donadel, and Federico Turrin. 2021. A Survey on Industrial Control System Testbeds and Datasets for Security Research. *IEEE Communications Surveys & Tutorials* 23, 4 (2021), 2248–2294. doi:10.1109/COMST.2021.3094360

[6] Martin Dobler, Michael Hellwig, Nuno Lopes, Ken Oakley, and Mike Winterburn. 2025. Systematic Review and Characterisation of Malicious Industrial Network Traffic Datasets. *International Journal of Information Security* 24, 5 (2025), 208. doi:10.1007/s10207-025-01126-9

[7] Danny Dolev and Andrew C. Yao. 1983. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* 29, 2 (1983), 198–208. doi:10.1109/TIT.1983.1056650

[8] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, New York, NY, USA, 1285–1298. doi:10.1145/3133956.3134015

[9] Mahmoud Elsisi, Marnel Altius, Shun-Feng Su, and Chun-Lien Su. 2023. Robust Kalman Filter for Position Estimation of Automated Guided Vehicles Under Cyberattacks. *IEEE Transactions on Instrumentation and Measurement* 72 (2023), 1–12. doi:10.1109/TIM.2023.3250285

[10] Jonathan Goh, Sridhar Adepu, Khurum Nazir Junejo, and Aditya Mathur. 2017. A Dataset to Support Research in the Design of Secure Water Treatment Systems. In *Critical Information Infrastructures Security (CRITIS) (LNCS, Vol. 10242)*. Springer, Cham, 88–99. doi:10.1007/978-3-319-71368-7_8

[11] Niv Goldenberg and Avishai Wool. 2013. Accurate Modeling of Modbus/TCP for Intrusion Detection in SCADA Systems. *International Journal of Critical Infrastructure Protection* 6, 2 (2013), 63–75.

[12] Hakan Kayan, Matthew Nunes, Omer Rana, Pete Burnap, and Charith Perera. 2022. Cybersecurity of Industrial Cyber-Physical Systems: A Review. *Comput. Surveys* 54, 11s (2022), 229:1–229:35. doi:10.1145/3510410

[13] Ognjen Kundacina, Vladimir Vincan, Gorana Gojic, Vukan Ninkovic, and Dragisa Miskovic. 2025. Conformal Anomaly Detection for Predictive Maintenance in Thermal Power Plants. *IEEE Access* 13 (2025), 39738–39752. doi:10.1109/ACCESS.2025.3546451

[14] Tze Leung Lai. 1998. Information Bounds and Quick Detection of Parameter Changes in Stochastic Systems. *IEEE Transactions on Information Theory* 44, 7 (1998), 2917–2929.

[15] Max Landauer, Sebastian Onder, Florian Skopik, and Markus Wurzenberger. 2023. Deep Learning for Anomaly Detection in Log Data: A Survey. *Machine Learning with Applications* 12 (2023), 100470. doi:10.1016/j.mlwa.2023.100470

[16] Shang Li, Yasin Yılmaz, and Xiaodong Wang. 2015. Quickest Detection of False Data Injection Attack in Wide-Area Smart Grids. *IEEE Transactions on Smart Grid* 6, 6 (2015), 2725–2735.

[17] Chih-Yuan Lin, Simin Nadjm-Tehrani, and Mikael Asplund. 2018. Timing-Based Anomaly Detection in SCADA Networks. In *Critical Information Infrastructures Security (CRITIS) (LNCS, Vol. 10707)*. Springer, Cham, 48–59. doi:10.1007/978-3-319-99843-5_5

[18] Qin Lin, Sridhar Adepu, Sicco Verwer, and Aditya Mathur. 2018. TABOR: A Graphical Model-based Approach for Anomaly Detection in Industrial Control Systems. In *Proceedings of the 2018 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*. ACM, New York, NY, USA, 525–536. doi:10.1145/3196494.3196546

[19] Hanxiao Liu, Yuqing Ni, Lihua Xie, and Karl Henrik Johansson. 2022. How Vulnerable is Innovation-Based Remote State Estimation: Fundamental Limits under Linear Attacks. *Automatica* 136 (2022), 110079. doi:10.1016/j.automatica.2021.110079

[20] Gary Lorden. 1971. Procedures for Reacting to a Change in Distribution. *The Annals of Mathematical Statistics* 42, 6 (1971), 1897–1908.

[21] Alexander Maier, Oliver Niggemann, Roman Just, Michael Jäger, and Asmir Vodenčarević. 2011. Anomaly Detection in Production Plants Using Timed Automata: Automated Learning of Models from Observations. In *Proceedings of the 8th International Conference on Informatics in Control, Automation and Robotics (ICINCO)*. SciTePress, Noordwijkerhout, The Netherlands, 363–369. doi:10.5220/0003538903630369

[22] Lukas Malburg, Joscha Grüger, and Ralph Bergmann. 2022. An IoT-Enriched Event Log for Process Mining in Smart Factories. arXiv preprint arXiv:2209.02702. doi:10.48550/arXiv.2209.02702 Dataset DOI: 10.6084/m9.figshare.20130794.

[23] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, and Rong Zhou. 2019. LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. International Joint Conferences on Artificial Intelligence Organization, Macao, China, 4739–4745. doi:10.24963/ijcai.2019/658

[24] Yilin Mo and Bruno Sinopoli. 2009. Secure Control Against Replay Attacks. In *Proceedings of the 47th Annual Allerton Conference on Communication, Control, and Computing*. IEEE, Monticello, IL, USA, 911–918. doi:10.1109/ALLERTON.2009.5394956

[25] László Monostori, Botond Kádár, Thomas Bauernhansl, Shinsuke Kondoh, Soundar Kumara, Gunther Reinhart, Olaf Sauer, Günther Schuh, Wilfried Sihn, and Kanji Ueda. 2016. Cyber-Physical Systems in Manufacturing. *CIRP Annals* 65, 2 (2016), 621–641. doi:10.1016/j.cirp.2016.06.005

[26] Arunava Naha, André Teixeira, Anders Åhlén, and Subhrakanti Dey. 2023. Quickest Detection of Deception Attacks on Cyber–Physical Systems with a Parsimonious Watermarking Policy. *Automatica* 155 (2023), 111147. doi:10.1016/j.automatica.2023.111147

[27] Timo Nolle, Stefan Luettgen, Alexander Seeliger, and Max Mühlhäuser. 2018. Analyzing Business Process Anomalies Using Autoencoders. *Machine Learning* 107 (2018), 1875–1893. doi:10.1007/s10994-018-5702-8

[28] Timo Nolle, Stefan Luettgen, Alexander Seeliger, and Max Mühlhäuser. 2022. BINet: Multi-Perspective Business Process Anomaly Classification. *Information Systems* 103 (2022), 101458. doi:10.1016/j.is.2019.101458

[29] Neha Rino and Thomas Chatain. 2024. Timed Alignments with Mixed Moves. *Data & Knowledge Engineering* 154 (2024), 102366. doi:10.1016/j.datak.2024.102366

[30] Ronny Seiger, Lukas Malburg, Barbara Weber, and Ralph Bergmann. 2022. Integrating Process Management and Event Processing in Smart Factories: A Systems Architecture and Use Cases. *Journal of Manufacturing Systems* 63 (2022), 575–592. doi:10.1016/j.jmsy.2022.05.012

[31] Glenn Shafer and Vladimir Vovk. 2008. A Tutorial on Conformal Prediction. *Journal of Machine Learning Research* 9 (2008), 371–421.

[32] Hyeok-Ki Shin, Woomyo Lee, Jeong-Han Yun, and HyoungChun Kim. 2020. HAI 1.0: HIL-Based Augmented ICS Security Dataset. In *Proceedings of the 13th USENIX Workshop on Cyber Security Experimentation and Test (CSET)*. USENIX Association, Berkeley, CA, USA, 5 pages. https://www.usenix.org/conference/cset20/presentation/shin

[33] Xiaobin Tan and Hongsheng Xi. 2008. Hidden Semi-Markov Model for Anomaly Detection. *Appl. Math. Comput.* 205, 2 (2008), 562–567. doi:10.1016/j.amc.2008.05.028

[34] Wil M. P. van der Aalst, Arya Adriansyah, and Boudewijn F. van Dongen. 2012. Replaying History on Process Models for Conformance Checking and Performance Analysis. *WIREs Data Mining and Knowledge Discovery* 2, 2 (2012), 182–192.

[35] Denis Volkhonskiy, Evgeny Burnaev, Ilia Nouretdinov, Alexander Gammerman, and Vladimir Vovk. 2017. Inductive Conformal Martingales for Change-Point Detection. In *Proceedings of the Sixth Workshop on Conformal and Probabilistic Prediction and Applications (Proceedings of Machine Learning Research, Vol. 60)*. PMLR, Stockholm, Sweden, 132–153.

[36] Vladimir Vovk, Ilia Nouretdinov, and Alex Gammerman. 2003. Testing Exchangeability On-Line. In *Proceedings of the Twentieth International Conference on Machine Learning (ICML)*, Tom Fawcett and Nina Mishra (Eds.). AAAI Press, Menlo Park, CA, 768–775.

[37] Shuaiqi Yuan, Jipu Li, Chunjin Wang, and Xiaoge Zhang. 2026. Conformal Machine Learning for Reliable Anomaly Detection in Industrial Cyber–Physical Systems. *Reliability Engineering & System Safety* 274 (2026), 112417. doi:10.1016/j.ress.2026.112417